

Predicting Patient Drug-Review Ratings

Prospective MScR Student: **Hana Esfandiar**

Supervisor: **Dr. Gal**

[Source Code](#)

Table of Content

1	Project Overview
2	Problem Statement & Objectives
3	Project Workflow
4	The Dataset
5	Exploratory Data Analysis (Notebook 01)
6	Data Preprocessing & Tokenization (Notebook 02)
7	Model Architecture & Training (Notebook 03)
8	Evaluation Metrics – Why Each One
9	Inference Pipeline (Notebook 04)
10	Interpretability & Explainability (Notebook 06)
11	Comparison with the Reference Paper
12	Generative-AI Task (Notebook 05)
13	Conclusion

1. Project Overview

This project builds a complete machine-learning system around the Drug Review Dataset, in which patients write free-text reviews of medications and assign a 1–10 star rating. The work has two deliverables, exactly as specified in the assignment:

(1) A deep-learning model that reads a review and predicts its 1–10 star rating. I framed this as a 10-class text classification problem and solved it with a Bidirectional LSTM network trained on 161,297 reviews and evaluated on a held-out test set of 53,766 reviews. Because the ratings are heavily imbalanced (31.6 % of all reviews are 10-star), I trained two versions of the model:

- a naïve baseline
- a class-weighted version

and analyzed the trade-off between them.

(2) A generative-AI track that for every review in the test set, uses a Large Language Model to produce a ten-word summary and a binary sentiment label. I implemented this with a locally hosted LLM (Llama 3.2 via Ollama) and experimented with two prompt engineering strategies:

- a zero-shot structured prompt
- a few-shot, system-instructed prompt

A third pillar of the project is interpretability: I used LIME, confusion-matrix analysis, error analysis and a confidence-calibration study to explain why the model makes the predictions it does, which is essential in a medical domain where blind trust in a black box is unacceptable.

Why this problem is genuinely difficult: Unlike binary sentiment classification (positive vs. negative), predicting a 10-point rating requires the model to resolve subtle linguistic gradations. The difference between a patient writing "it helped somewhat" (Rating 5) versus "it worked well enough" (Rating 7) is a matter of degree, not direction. Additionally, the dataset is severely class-imbalanced, with Rating 10 accounting for over 31% of all reviews while Rating 4 accounts for only 3.1%. This imbalance requires deliberate corrective measures during training.

2. Problem Statement & Objectives

The assignment defines three concrete tasks. I restate them together with how interpreted each one, because the framing decisions drive every later choice.

2.1 Task definitions

	Task	My Interpretation
1	Build a deep-learning model on the train set to predict the patient rating on the test set.	A supervised 10-class classification problem: input = review text, output = one of the ratings 1-10. Reported with established classification metrics.
2	Report results, provide interpretability/explainability, and compare to the dataset paper.	Classification report + confusion matrix + ordinal error metrics, then LIME word-level explanations and an explicit comparison section.
3	Use a generative-AI model to summarize and classify the sentiment of every test review; deliver an Excel file; experiment with multiple prompting approaches; supply two output files.	A separate LLM inference pipeline producing a ≤10-word summary and a positive/negative label, with two distinct prompt designs exported as two spreadsheets.

WHY CLASSIFICATION AND NOT REGRESSION?

The rating is technically a number, so regression (predicting a continuous value) is tempting. I chose classification as the primary framing because the assignment literally asks to "predict patient rating" as a label, the ratings are discrete integers, and classification gives me a full per-class breakdown (precision/recall/F1 per star) that exposes exactly where the model fails. I nonetheless borrow one regression style metric, Mean Absolute Error, to respect the fact that the labels are ordinal (a 9 predicted as 10 is a much smaller mistake than a 1 predicted as 10). I will discuss this hybrid view in Section 8.

2.2 Success criteria

Given the difficulty and imbalance of the data, I defined success not as a single accuracy number but as:

- A model that demonstrably learns real sentiment signal
- An honest, metric-rich evaluation that does not hide failure modes
- Explanations that a clinician could inspect
- A reproducible generative pipeline with well-engineered prompts.

3. Project Workflow

Before diving into any code, it helps to see the whole pipeline. I deliberately split the work into six self-contained notebooks, each producing artefacts (cleaned data, a trained model, a vocabulary file, figures, spreadsheets) that the next stage consumes. This separation mirrors a real production ML pipeline: each step can be rerun independently, and the saved artefacts make the results reproducible without retraining every time.

Stage	Notebook	Input	Output	Purpose
Explore	01_EDA	Raw CSVs	Figures, statistics	Understand distributions; decide hyper-parameters (sequence length, metrics).
Preprocess	02_Preprocess	Raw CSVs	Processed_data.npz , vectorizer_vocab.json	Clean text, tokenise, pad to fixed-length integer matrices.
Train	03_Training	processed_data.npz	Two .keras models	Train baseline + class- weighted Bi-LSTM; compute classification reports.
Infer	04_Inference	saved model + vocab	Live predictions	Reusable function that scores any custom review.
Generate	05_generative_AI	test set + LLM	2 Excel files	LLM summary + sentiment under two prompt strategies.
Explain	06_Interpretability	saved model + test set	LIME & error figures	Explain model behaviour and diagnose failures.

The vocabulary is exported to JSON in stage 2 and re-loaded in stages 4–6. This guarantees that training and inference use exactly the same word-to-integer mapping. A mismatch here is one of the most common silent bugs in NLP pipelines (the model would receive integers that mean something different from what it was trained on).

4. The Dataset

The dataset is the UCI Drug Review Dataset, originally collected from Drugs.com and made publicly available through a Kaggle competition. It contains patient reviews of prescription and over-the-counter medications, each labeled with the medical condition being treated and a 10-star satisfaction rating provided by the patient. The data ships as two CSV files inside a single archive: a training split and a test split. The split is provided by the dataset authors, so I keep it intact, which keeps my evaluation honest and comparable to other work on this dataset.

Column	Type	Meaning	Role in this project
<code>uniqueID</code>	int	Row identifier	Ignored (no predictive value).
<code>drugName</code>	text	Name of the medication	Carried into the Excel deliverable; not fed to the model.
<code>condition</code>	text	Medical condition treated	Carried into the Excel deliverable.
<code>review</code>	text	Free-text patient review	The single model input.
<code>rating</code>	int 1–10	Patient's star rating	The prediction target.
<code>date</code>	text	Review date	Ignored.
<code>usefulCount</code>	int	How many users found it useful	Ignored (would leak post-hoc popularity).

Split	Rows	Missing <code>condition</code>	Missing <code>review</code> / <code>rating</code>
Train	161,297	899 (0.56 %)	0
Test	53,766	295 (0.55 %)	0

WHY I DELIBERATELY USE ONLY THE REVIEW TEXT

I made a conscious decision to feed the model only the free-text review, even though *drugName* and *condition* are available. Two reasons:

- 1. the assignment is fundamentally a natural language understanding task, so the point is to read sentiment from prose
- 2. including *usefulCount* in particular would be data leakage, because that count is accumulated after the review is posted and correlates with rating in a way the model could exploit without learning anything linguistic.

5. Exploratory Data Analysis

The goal of this stage was to answer three questions:

- Is the data clean?
- Is the target balanced?
- How long is a typical review?

Every later hyper-parameter (sequence length, choice of metrics, decision to use class weights) is justified by something I found here.

5.1 Data loading & integrity checks

I load both CSVs directly from the zip archive and immediately check shapes, data types and missing values. This is the cheapest possible insurance against a corrupt download.

01_EDA.ipynb — loading & null-checking

```
with zipfile.ZipFile(zip_path, 'r') as zip_file:
    df_train = pd.read_csv(zip_file.open('drugsComTrain_raw.csv'))
    df_test = pd.read_csv(zip_file.open('drugsComTest_raw.csv'))

print(df_train.shape)          # (161297, 7)
print(df_train.isnull().sum()) # condition: 899 missing, everything else 0
```

What the output tells me is that the only missing values are in **condition** (899 train / 295 test). Critically, no review and no rating is ever missing. The two columns I actually need are 100 % complete. That single fact means I can drop the few **condition** nulls without losing any usable training signal.

5.2 Rating distribution & class imbalance

This is the most consequential plot in the entire project. I counted how many reviews fall into each star rating.

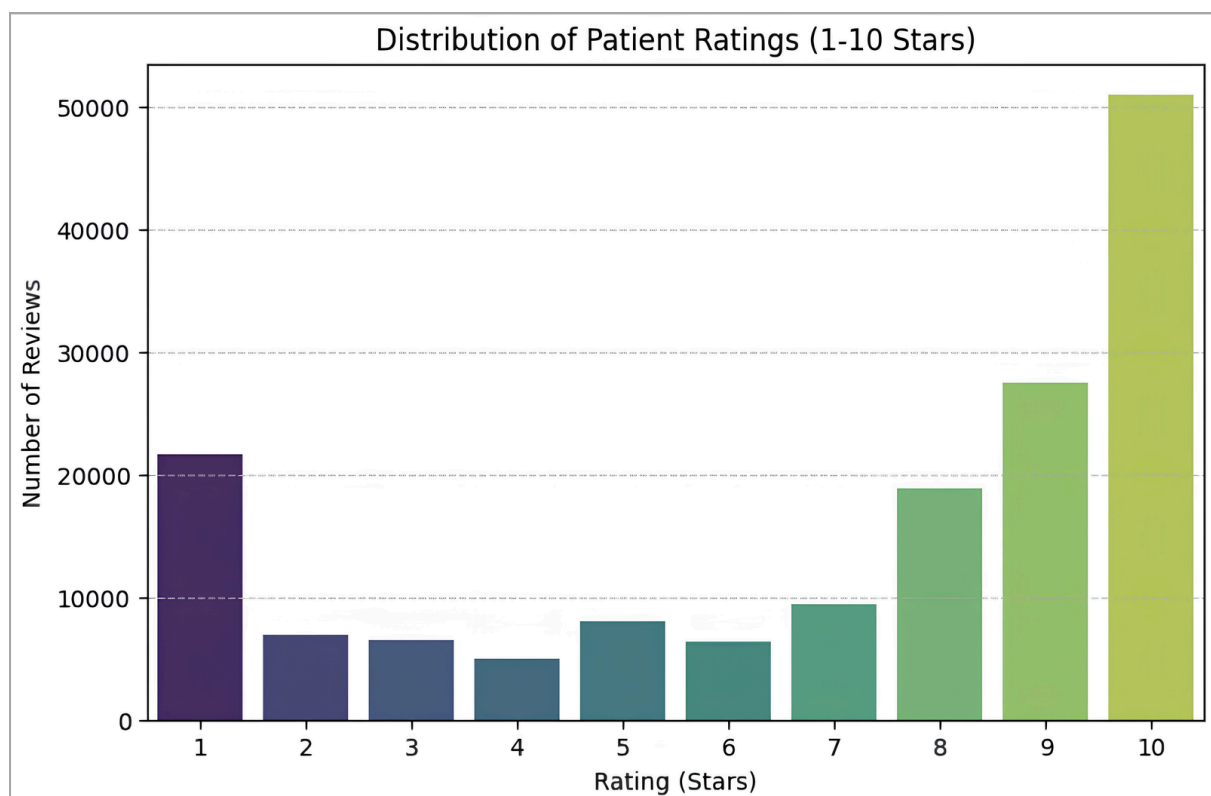


Figure 1. Distribution of the 1-10 star ratings across the 161,297 training reviews. The data is strongly skewed toward the positive extreme.

Rating	1	2	3	4	5	6	7	8	9	10
Count	21,619	6,931	6,513	5,012	8,013	6,343	9,456	18,890	27,531	50,989
Share	13.4%	4.3%	4.0%	3.1%	5.0%	3.9%	5.9%	11.7%	17.1%	31.6%

Key finding: the distribution is U-shaped and severely imbalanced. Rating 10 alone is 31.6 % of the data, and the two extremes (1 and 10) together make up 45 %. The "middle" ratings (3–6) are each only 3–5 %. Patients tend to review a drug when they either love it or hate it, and neutral experiences are under-reported.

WHY THIS DRIVES THE REST OF THE PROJECT

This single chart forecasts two things that turn out to be true. (1) Plain accuracy will be a misleading metric, which means a lazy model could predict "10" every time and still be right 31.6 % of the time without learning anything. That is exactly why I report per-class F1 and a confusion matrix rather than only accuracy. (2) The minority middle-classes will be starved of examples, so I anticipated needing class weights to stop the network from ignoring them.

5.3 Review length analysis

A neural network needs a fixed input length, so I must pick a cut-off. If I choose the length too short, I'll truncate meaning and if I choose the length too long, I'll waste memory and compute padding mostly-empty sequences. I made this an evidence-based decision by studying the word-count distribution.

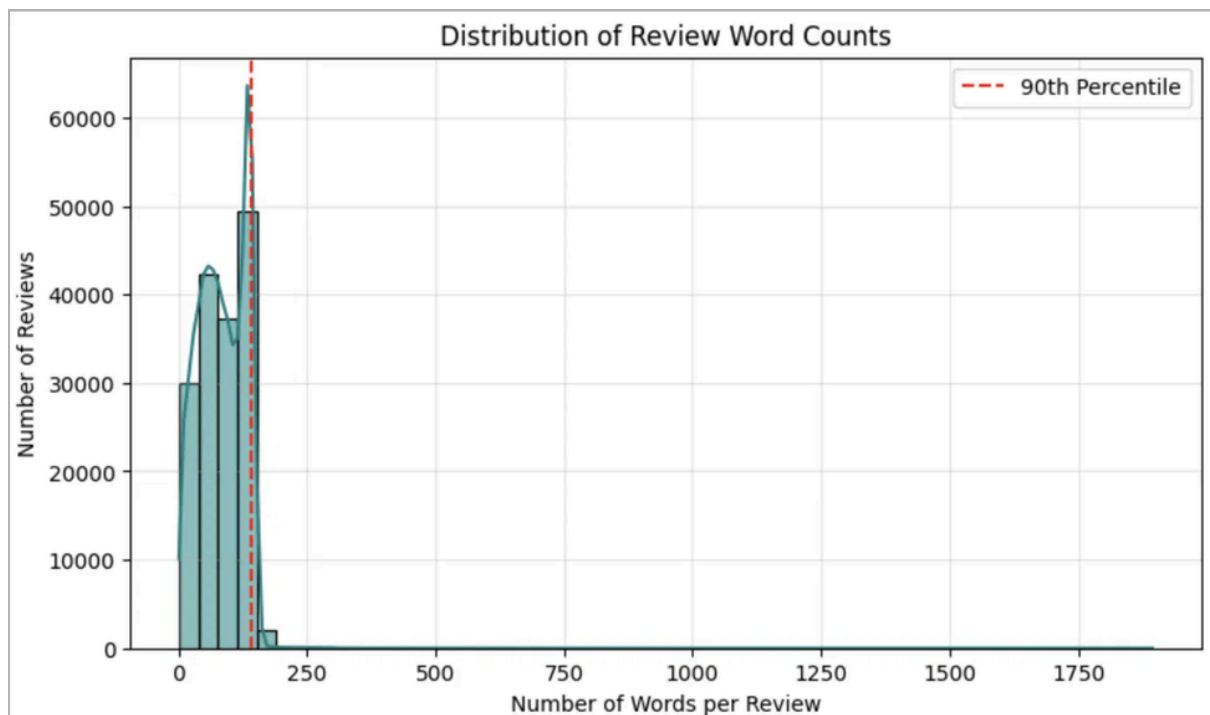


Figure 2. Distribution of review word counts. The red line marks the 90th percentile (≈ 141 words). A small number of outliers stretch to nearly 1,900 words.

WHY I CHOSE A SEQUENCE LENGTH OF 150

The median review is 84 words and the 95th percentile is 146. I set the model's `MAX_LEN = 150`. This covers more than 95 % of reviews in full, so I lose almost no information, while ignoring the handful of 1,000+ word outliers that would otherwise force every single review to be padded with hundreds of meaningless zeros, which is a huge, wasteful increase in compute for the entire dataset to accommodate a few rare cases. 150 is the data-justified sweet spot between coverage and efficiency.

6. Data Preprocessing & Tokenization

Raw text from a web platform like Drugs.com is noisy. It contains HTML encoding artefacts, inconsistent casing, and trailing whitespace. These inconsistencies fragment the vocabulary. For example, the model would treat "Pain", "pain", and "PAIN" as three different tokens even though they carry identical meaning. Preprocessing standardizes the text so the model can focus purely on semantic content.

6.1 Text cleaning

02_preprocessing.ipynb — cleaning function

```
def clean_text(text):
    if not isinstance(text, str):
        return ""
    text = html.unescape(text)  # turn &#039; back into a real apostrophe
    text = text.lower()        # "Amazing" and "amazing" become one token
    text = text.strip()        # remove leading/trailing whitespace
    return text
```

WHY THIS CLEANING IS DELIBERATELY LIGHT?

I keep punctuation, stop-words and negations. This is intentional, because classic text-mining pipelines aggressively strip stop words, but in sentiment tasks words like "not", "no" and "but" are the single most important tokens, "not effective" is the opposite of "effective". Over-cleaning would destroy exactly the signal I need. The `html.unescape` step is the one piece of cleaning that is unambiguously beneficial, because raw HTML artefacts carry no meaning at all.

6.2 Tokenization & vocabulary

I convert words into integers using Keras's modern **TextVectorization** layer. It learns a vocabulary from the training text and maps each review to a fixed-length integer vector.

02_preprocessing.ipynb — vectorisation

```
VOCAB_SIZE = 10000 # keep only the 10k most frequent words
MAX_LEN     = 150   # the length justified by the EDA in Section 5.3

vectorize_layer = TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_mode='int',
    output_sequence_length=MAX_LEN)

vectorize_layer.adapt(train_text_ds) # build vocabulary FROM TRAIN ONLY
X_train_padded = vectorize_layer(df_train['clean_review']).numpy() # (161297, 150)
X_test_padded  = vectorize_layer(df_test['clean_review']).numpy()  # (53766, 150)
```

WHY 10,000 WORDS, AND WHY "ADAPT" ONLY ON TRAIN

A 10k vocabulary captures the overwhelming majority of word occurrences while discarding rare tokens (typos, drug brand names seen once) that the model could never learn from anyway and that would only bloat the embedding table. Equally important, I call `.adapt()` on the training data only. If I had built the vocabulary from the combined train+test text, information about the test set would leak into the model's design, a subtle but real form of cheating that inflates reported scores. Building the vocabulary on train alone keeps the test set a true unseen hold-out.

The processed arrays and the vocabulary are saved (**processed_data.npz**, **vectorizer_vocabulary.json**) so every downstream notebook reuses an identical, frozen representation. The ratings are saved as-is (1–10) and shifted to 0–9 only at training time, because neural-network class indices must start at zero.

7. Model Architecture & Training

In this phase, the model must read a sequence of 150 integers and output a probability for each of the 10 possible ratings.

7.1 Why a Bidirectional LSTM

Text classification can be approached with many architectures. The choice of a Bidirectional LSTM (Bi-LSTM) is deliberate and justified on multiple grounds:

- **Sequential context:** Sentiment in drug reviews is context-dependent. The word "pain" in "it eliminated my pain" carries a positive signal, while the same word in "it caused me pain" is negative. An LSTM processes the sequence token-by-token, maintaining a hidden state that captures this evolving context, which is something a bag-of-words model cannot do.
- **Bidirectionality:** A standard LSTM only reads text left-to-right. A Bidirectional LSTM runs two LSTMs simultaneously, one forward and one backward and concatenates their outputs. This allows the model to use both past and future context when encoding each word, which is particularly useful for negation patterns like "not bad at all".
- **Efficiency:** Compared to Transformer-based architectures, an LSTM is significantly more lightweight. For a 10-class ordinal classification task on a dataset of 160,000 samples, a Bi-LSTM provides a strong baseline without requiring GPU clusters or days of pretraining.

03_Training.ipynb — model definition

```
model = Sequential([
    Embedding(input_dim=VOCAB_SIZE, output_dim=64), # word IDs -> 64-dim meaning vectors
    Bidirectional(LSTM(64)), # read the sentence both directions
    Dropout(0.5), # regularisation
    Dense(10, activation='softmax') # one probability per rating 1-10
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

7.2 Architecture

I chose a three-part architecture: an **Embedding** layer, a **Bidirectional LSTM**, and a **Dense softmax** output.

Layer	What it does	Why I chose it
Embedding (10000 → 64)	Maps each word ID to a dense 64-dimensional vector that the network learns.	Lets semantically similar words ("great", "wonderful") sit near each other in vector space — far richer than one-hot encoding.
Bidirectional LSTM (64 units)	An RNN that processes the review word-by-word, keeping a memory. "Bidirectional" runs it forward <i>and</i> backward.	Sentiment depends on word <i>order</i> and long-range context ("no side effects" vs "side effects"). Reading both directions captures cues that only become clear at the end of a sentence.
Dropout (0.5)	Randomly silences half the neurons each training step.	The dataset is large and the model could memorise quirks; aggressive dropout forces it to learn robust, general patterns.
Dense softmax (10)	Outputs 10 probabilities that sum to 1.	The natural output for single-label multi-class classification.

WHY AN LSTM RATHER THAN A SIMPLER OR A HEAVIER MODEL

A bag-of-words + logistic-regression baseline (what the original paper used) ignores word order entirely. A full Transformer (e.g. BERT) would likely score higher but needs far more compute and would obscure the educational point. The Bi-LSTM is the sweet spot for this assignment: it is a genuine deep-learning sequence model that respects word order and context, trains in minutes on a CPU, and at ~2.1M parameters is small enough to interpret cleanly with LIME.

7.3 The baseline model and the imbalance trap

I first trained the network straightforwardly, with **EarlyStopping** (patience 2, restoring the best weights) to halt once validation loss stopped improving, a guard against **overfitting** and wasted compute.

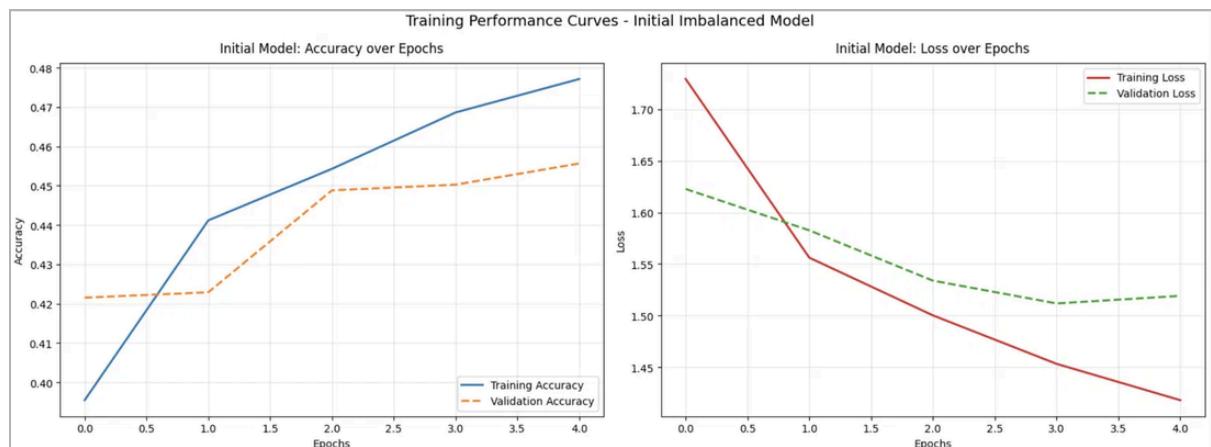


Figure 3. Baseline model training curves. Training accuracy keeps rising while validation accuracy plateaus near 0.46 and validation loss flattens/ticks up, the classic onset of overfitting that EarlyStopping is designed to catch.

The baseline reaches ~45 % accuracy, which sounds acceptable until you read the per-class report:

03_Training.ipynb — baseline classification report (abridged)

	precision	recall	f1-score	support	
Rating 1	0.52	0.80	0.63	7299	
Rating 2	0.00	0.00	0.00	2334	<- never predicted
Rating 4	0.00	0.00	0.00	1659	<- never predicted
Rating 6	0.00	0.00	0.00	2119	<- never predicted
Rating 10	0.57	0.81	0.67	17016	
accuracy			0.45	53766	
macro avg	0.22	0.25	0.22	53766	

The trap revealed: the baseline achieves its 45 % accuracy by almost entirely ignoring the minority classes, ratings 2, 4 and 6 receive an F1 of 0.00 because the

model never predicts them at all. It has learned the lazy strategy the EDA warned about: lean on the popular extremes. The gap between 45 % accuracy and a macro-F1 of just 0.22 is the quantitative fingerprint of this failure.

7.4 The class-weighted (balanced) model

To force the network to care about rare ratings, I computed balanced class weights, making a mistake on a rare class cost proportionally more in the loss function and retrained from scratch.

03_Training.ipynb — class weighting

```
class_weights = compute_class_weight('balanced',
                                     classes=np.unique(y_train_shifted), y=y_train_shifted)
# Rating 4 (rarest) -> weight 3.22 ; Rating 10 (commonest) -> weight 0.32
balanced_model.fit(X_train, y_train_shifted, class_weight=class_weight_dict, ...)
```



Figure 4. Class-weighted model training curves. Note the weighted loss decreases smoothly for both train and validation; absolute accuracy is lower than the baseline precisely because the model is no longer allowed to coast on the majority class.

The trade-off, quantified. Class weighting lowers headline accuracy (45 % → 38 %) but raises macro-F1 (0.22 → 0.28) and, crucially, gives every previously ignored class a non-zero F1. The minority ratings 2, 4 and 6 went from 0.00 to 0.20, 0.21 and 0.18. The model is now a worse "guesser" of the dominant class but a far more honest and useful classifier across the full scale.

Model	Accuracy	Macro F1	Weighted F1	Minority classes predicted?
Baseline (unweighted)	0.45	0.22	0.39	No — F1=0 for ratings 2,4,6
Class-weighted (chosen)	0.38	0.28	0.39	Yes — all 10 classes predicted

WHY I SELECT THE LOWER-ACCURACY MODEL?

This is the central modelling judgement of the project, and I want it to be explicit. A model that refuses to ever output "rating 4" is useless for the assignment's actual goal of predicting across the whole 1–10 scale, regardless of its accuracy headline. I therefore carry the class-weighted model forward into inference, interpretability and the generative pipeline.

8. Evaluation Metrics

Evaluation is performed on the held-out test set of 53,766 reviews. Multiple metrics are reported to capture different facets of model behavior. Accuracy alone is an insufficient metric for this task due to class imbalance. Here is the reasoning behind every metric in the report.

Metric	What it measures	Why I use it here
Accuracy	Fraction of exactly-correct predictions.	Reported for completeness and intuition, but explicitly <i>trusted</i> as a primary metric because of the imbalance (a "predict 10 always" model scores 31.6 %).
Precision / Recall	Of predicted-X, how many were right / of true-X, how many were found.	Separates the two ways a classifier can fail. Recall per class reveals which ratings the model misses entirely.
F1 score (per class)	Harmonic mean of precision and recall.	My headline metric. A single number that is only high when <i>both</i> precision and recall are good, so it cannot be gamed by the majority-class shortcut that fools accuracy.
Macro-F1	Unweighted average of per-class F1.	Treats every rating as equally important regardless of size — the right lens when I care about minority classes.
Weighted-F1	Support-weighted average of per-class F1.	Reflects real-world performance where most reviews are positive.
Mean Absolute Error	Average distance $ \text{true} - \text{predicted} $ in stars.	Respects the <i>ordinal</i> nature of ratings: predicting 9 for a true 10 is nearly right, predicting 1 is badly wrong — a distinction F1 cannot see.
Confusion matrix	Full table of true vs predicted classes.	Shows the <i>structure</i> of errors (are mistakes local or wild?), which drives the error analysis in Section 10.

WHY F1 OVER PLAIN ACCURACY

On an imbalanced dataset, accuracy answers the wrong question. It rewards a model for being right on the easy, common cases and is almost blind to whether the model can handle rare cases at all. F1, computed per class and then macro-averaged, forces the model to prove competence on every rating. This is precisely why my class-weighted model, which has lower accuracy but higher macro-F1, is the better model for this task.

WHY I ADD MAE

Standard classification metrics treat the ten ratings as ten unrelated categories, as if confusing "9" with "10" were as bad as confusing "1" with "10". But ratings are ordered numbers. MAE captures this: my model's MAE of 1.52 stars means predictions are typically within one-and-a-half stars of the truth, which is a genuinely useful result that the 38 % accuracy figure completely hides. Using MAE alongside F1 shows I understand that the target is ordinal, not merely categorical.

9. Inference Pipeline

A trained model is only useful if it can score new, unseen reviews. This notebook rebuilds the exact preprocessing used in training and wraps it in a single reusable function. The key engineering detail is that I reconstruct the **TextVectorization** layer from the saved vocabulary JSON, guaranteeing the integer mapping matches training.

04-Inference.ipynb — production prediction function

```
inference_vectorizer = TextVectorization(max_tokens=10000, output_mode='int',
    output_sequence_length=150, vocabulary=saved_vocabulary) # <- frozen mapping

def predict_custom_review(review_text):
    cleaned = clean_text(review_text)
    vec = inference_vectorizer([cleaned]).numpy()
    probs = model.predict(vec, verbose=0)
    return np.argmax(probs, axis=1)[0] + 1 # shift 0-9 back to human 1-10
```

I sanity-checked the pipeline on hand-written reviews spanning the sentiment spectrum. The results are exactly what we would hope:

Hand-written review	Predicted	Sensible?
"This medication completely changed my life... No side effects at all!"	10 / 10	✓ Strong positive → top rating
"Absolutely horrible experience. I suffered terrible nausea and severe headaches..."	1 / 10	✓ Strong negative → bottom rating
"The drug works well enough... but the fatigue makes it almost impossible to stay awake."	5 / 10	✓ Genuinely mixed → middle rating
"It was okay. It did the job, but nothing special. Average results."	4 / 10	✓ Lukewarm → low-middle

These qualitative checks confirm the model behaves rationally at the level of individual reviews before I subject it to the rigorous statistical and interpretability analysis that follows.

10. Interpretability & Explainability

The assignment requires interpretability, and in a medical context it is non-negotiable: a decent test score is not enough, so I must demonstrate that the model learns genuine clinical language rather than exploiting accidental artefacts. I analyzed this from four angles:

- per-class performance
- the confusion matrix
- LIME word-level explanations
- An error/confidence study

10.1 Per-class performance

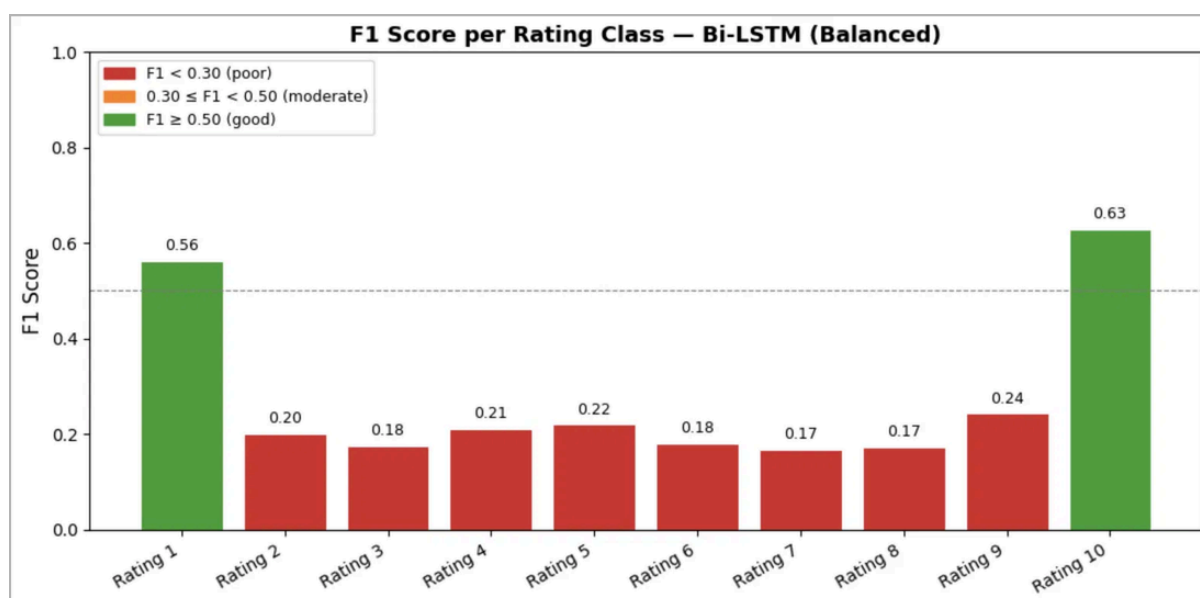


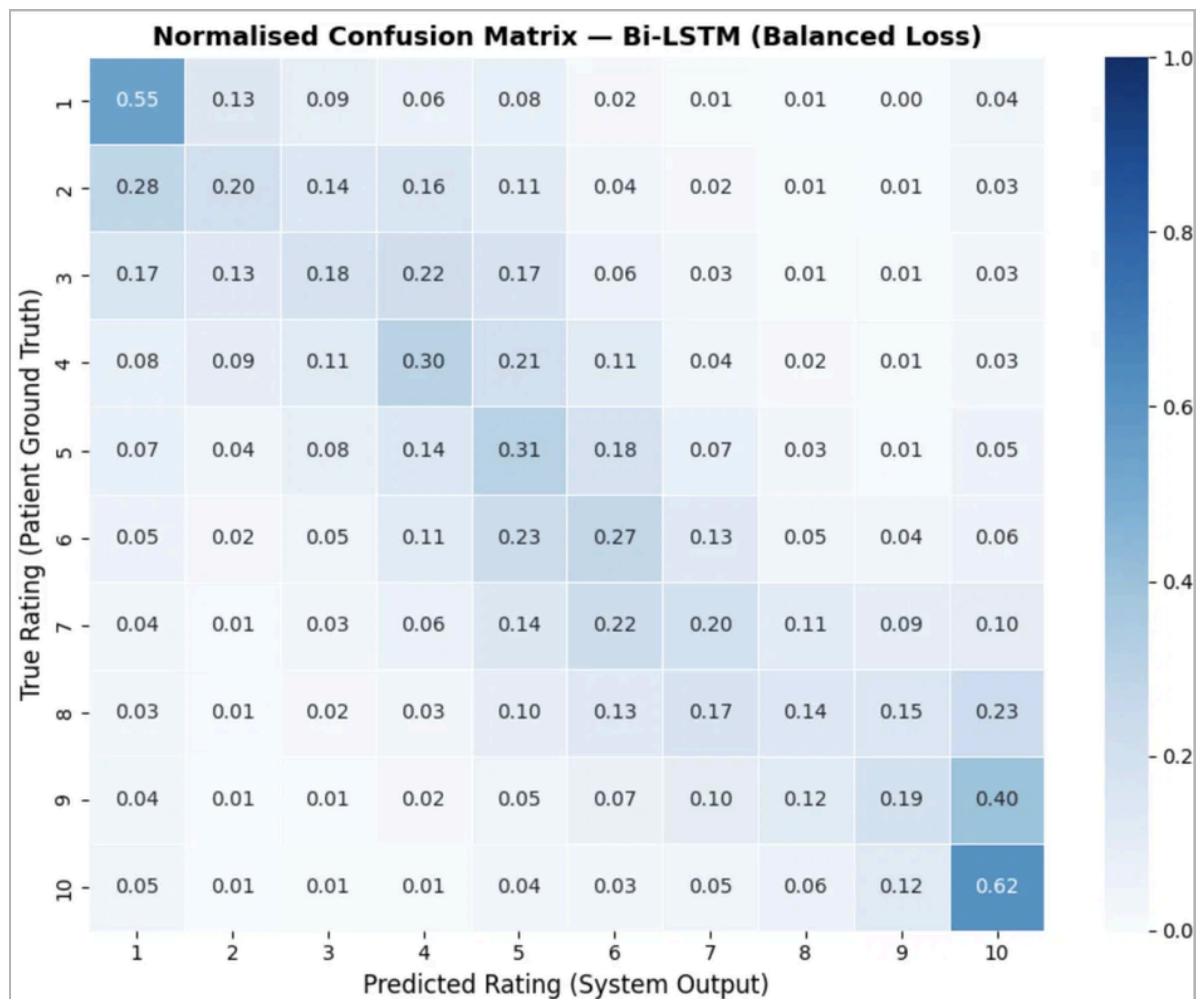
Figure 5. F1 score for each rating. Only the extremes (ratings 1 and 10) clear the 0.5 line; the middle of the scale is where the model struggles.

The model is good at the poles and weak in the middle. Ratings 1 (F1 0.56) and 10 (F1 0.63) are reliably identified; ratings 3–8 hover around 0.17–0.22. This is not a bug so much as a reflection of the language itself, a 1-star review is full of unambiguous outrage, whereas a 5-star review mixes praise and complaint, which is intrinsically harder to pin to one exact number.

10.2 Confusion matrix analysis

The errors are mostly local, which is good news. The mass sits near the diagonal: rating 9 is most often confused with 10 (39.5 % of true-9s), rating 6 with 5, rating 3 with 4. The model rarely makes wild errors; it mostly disagrees by one or two stars. This local-error structure is exactly why MAE (1.52) is low even though

exact accuracy is modest, and it is strong evidence that the model has learned a coherent sentiment ordering, not random associations.



10.3 LIME local explanations

LIME (Local Interpretable Model-agnostic Explanations) explains a single prediction by perturbing the review, randomly deleting words, running the perturbed versions through the model, and fitting a simple linear model to see which words actually moved the prediction. Green bars push toward the predicted rating; red bars push against it.

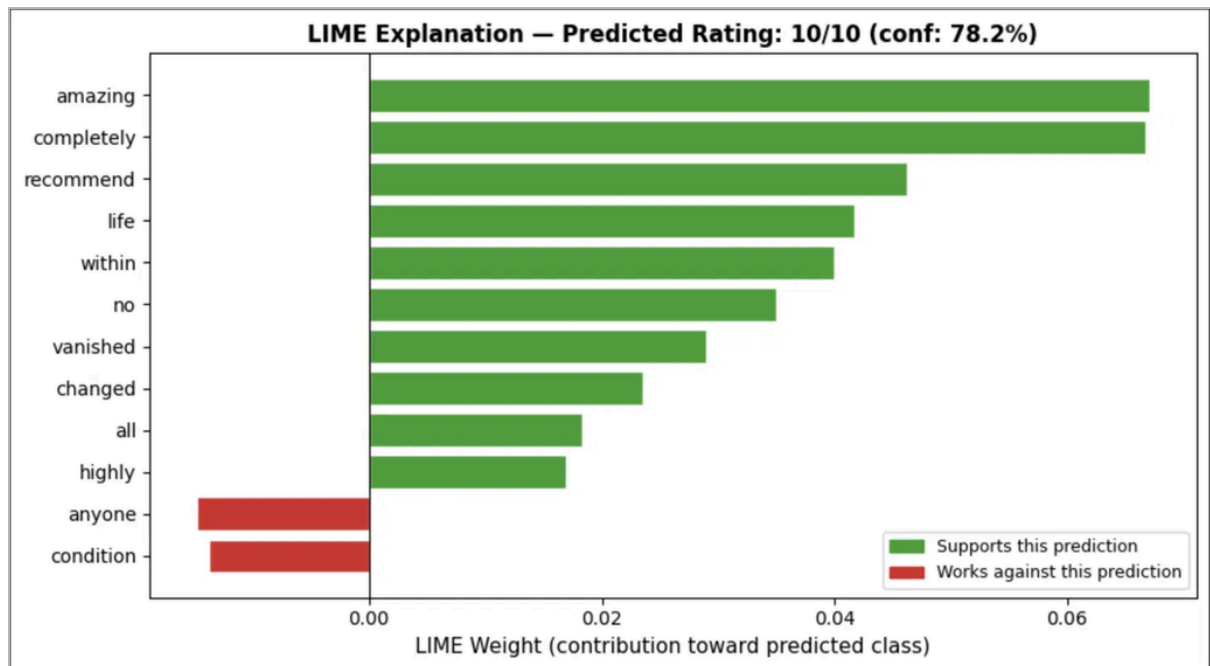


Figure 7. Clearly positive review → predicted 10/10 (78 % confidence). The drivers are "amazing", "completely", "recommend", "life", "vanished", "changed" exactly the vocabulary a human associates with a glowing review.

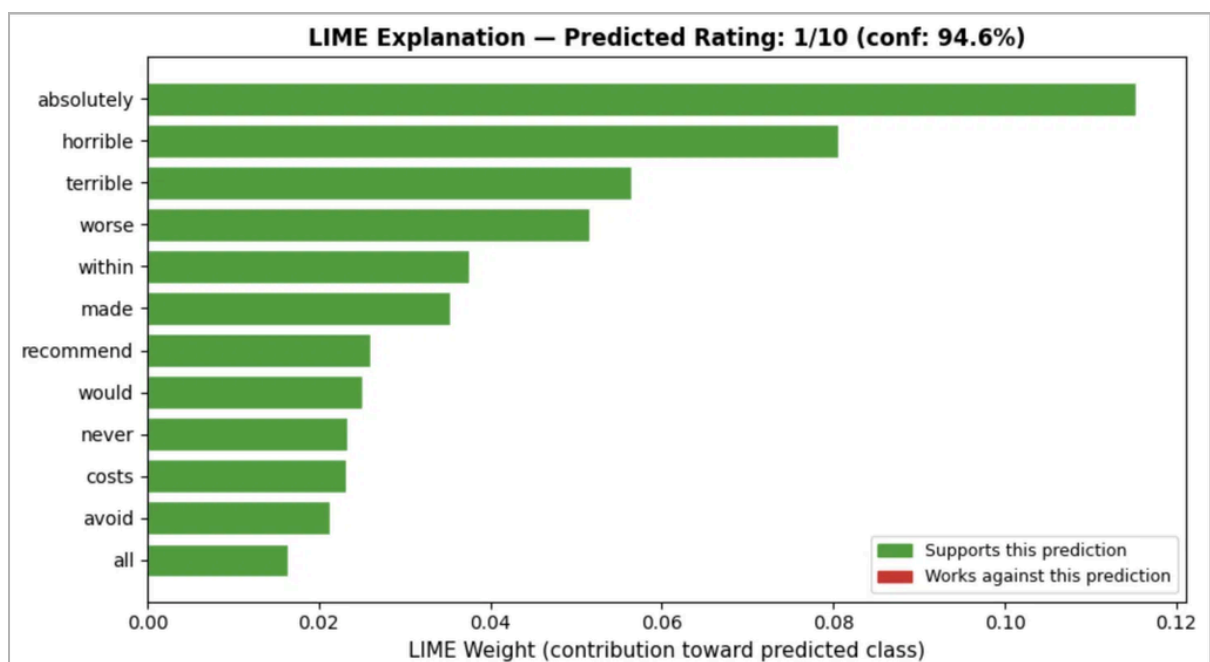


Figure 8. Clearly negative review → predicted 1/10 (95 % confidence). The drivers are "absolutely", "horrible", "terrible", "worse", "avoid", "never" genuine negative-sentiment words. Note the very high confidence.

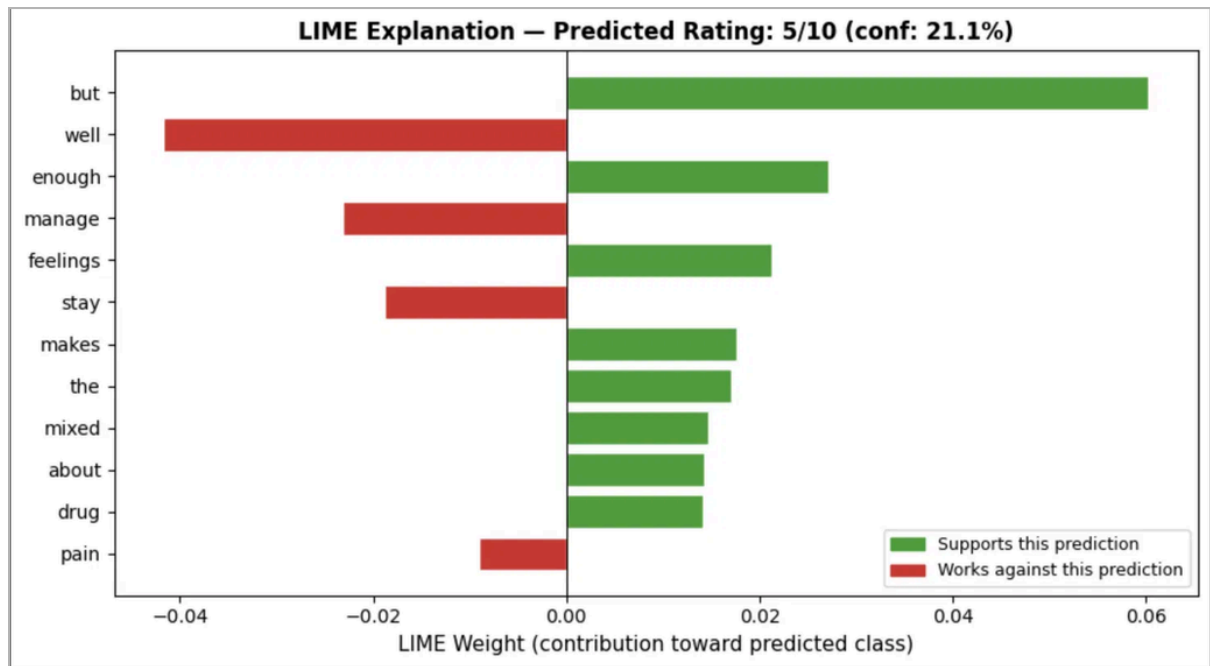


Figure 9. Ambiguous review → predicted 5/10 but at only 21 % confidence. The contrastive word "but" pushes toward a middle rating while "well" pushes against it. The model is appropriately uncertain on a genuinely mixed review.

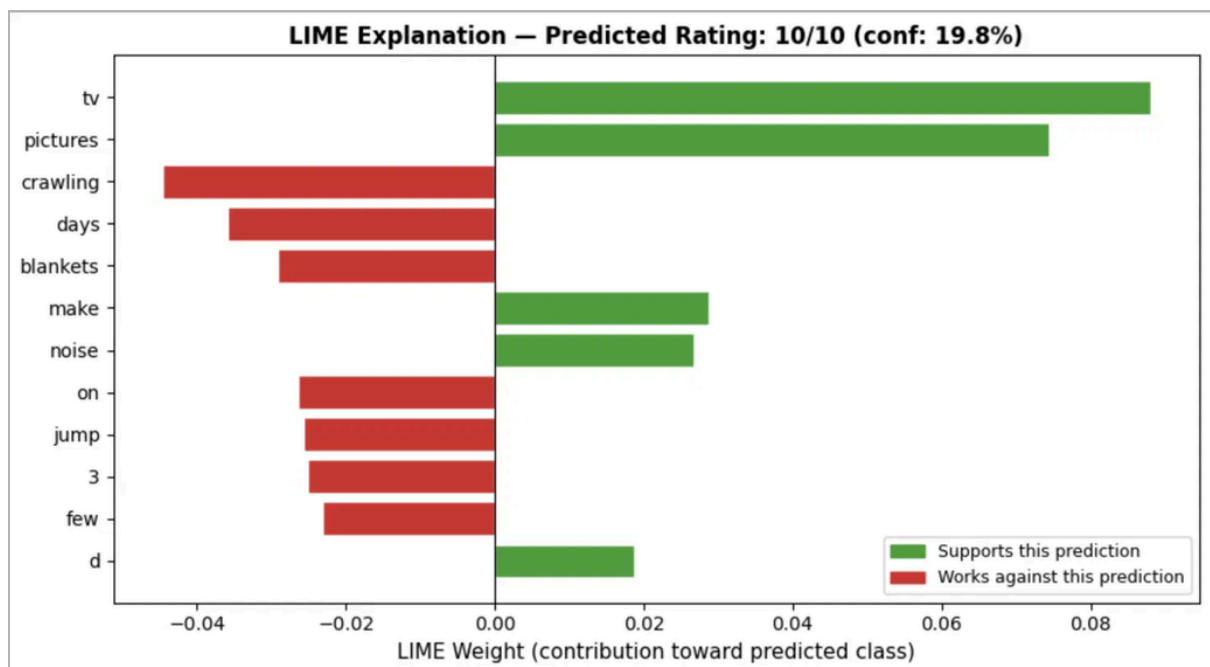


Figure 10. A real failure: true rating 1, predicted 10, but at just 20 % confidence. The model latched onto neutral nouns ("tv", "pictures") while the genuinely negative words ("crawling") correctly pushed against the wrong prediction. Low confidence flags this as a guess, not a conviction.

10.4 Error analysis & confidence calibration

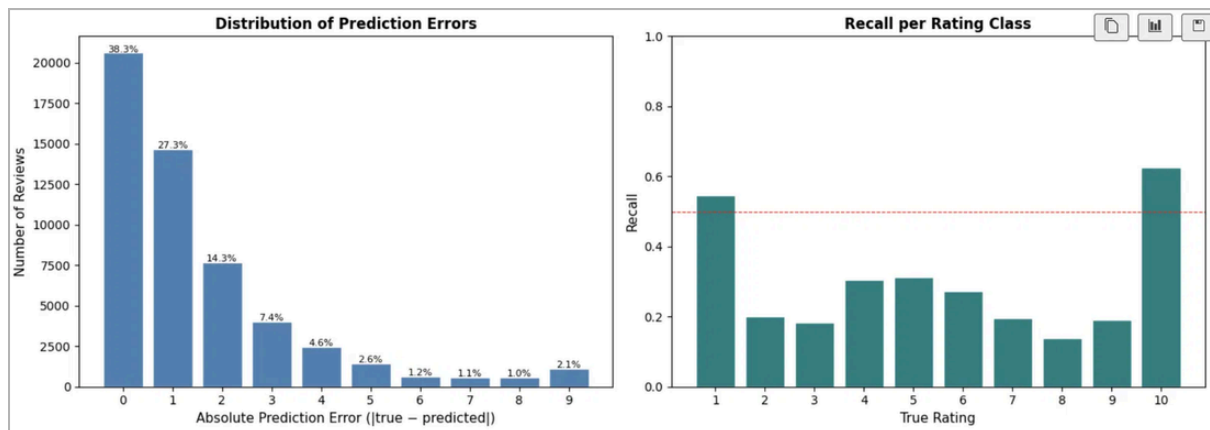


Figure 11. Left: distribution of absolute prediction error, 38 % exact, 27 % off by one, errors fall off steeply. Right: recall per class, confirming the U-shaped competence (strong at 1 and 10, weak in the middle).

Within ± 0 (exact)	Within ± 1 star	Within ± 2 stars	Off by ≥ 5 stars
38.3 %	65.6 %	79.9 %	8.1 %

Four out of five predictions are within two stars of the patient's true rating, and catastrophic errors (≥ 5 stars off) are rare at 8 %. For a 10-way ordinal problem on noisy free text, this is a respectable result that the bare accuracy number badly undersells.

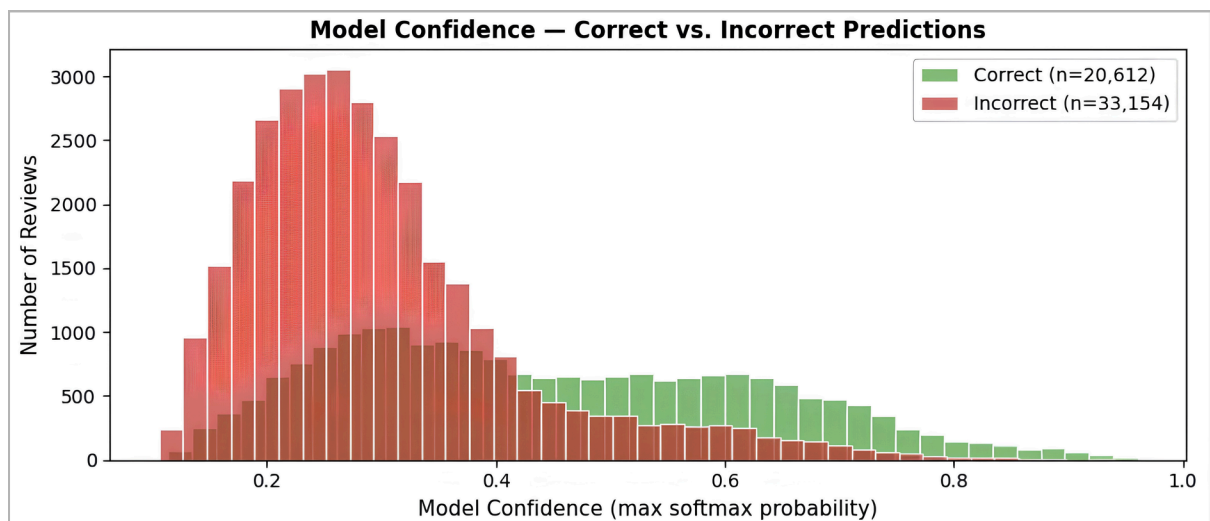


Figure 12. Softmax confidence on correct (green) vs incorrect (red) predictions. Correct predictions are, on average, more confident (0.45) than incorrect ones (0.30) but the distributions overlap heavily.

11. Comparison with the Reference Paper

The dataset was published with the paper " *Aspect-Based Sentiment Analysis of*

Drug Reviews Applying Cross-Domain and Cross-Data Learning, " Proceedings of the 2018 International Conference on Digital Health (DH '18). A fair comparison requires first understanding that they solved a different, easier problem.

11.1 What the paper actually did

Dimension	Gräßer et al. (2018)	This project
Target	Aspect-based sentiment polarity for overall satisfaction, effectiveness, and side effects	Exact 1–10 star rating (10 classes)
Class count	3 (disjoint classes): negative, neutral, positive	10
Model family	Classical ML — Logistic Regression using n-gram features	Deep learning — Bidirectional LSTM over learned embeddings
Headline result	92.24% accuracy achieved for the 3-class overall satisfaction task on the Drugs.com dataset	0.38 exact accuracy / 0.28 macro-F1 (10-class)
Extra focus	Cross-domain and cross-data transfer learning, investigating how models trained on one medical condition or website perform on others	Interpretability (LIME) + a generative-AI track

WHY THE RAW NUMBERS (0.92 VS 0.38) MUST NOT BE COMPARED DIRECTLY

A 2-class problem has a 50 % random baseline; a 10-class problem has a 10 % random baseline. Distinguishing "positive vs negative" is something even a keyword counter does well, whereas distinguishing "a 7 from an 8" is hard even for a human reading the same text. Quoting "92 % vs 38 %" as if my model were worse would be a misleading comparison of two fundamentally different tasks. The intellectually honest comparison is to put my model on their footing.

A direct numerical comparison is not meaningful because the two works solve fundamentally different versions of the problem. Collapsing 10 ratings into 3 classes inherently makes the classification task easier: all linguistic variation within the "positive" range (Ratings 7 through 10) becomes irrelevant noise that the model can ignore. Our model must discriminate between Rating 7 and Rating 8, distinctions that are often invisible at the sentence level.

To enable a fair comparison, if our model's 10-class predictions are collapsed to 3 classes using the same thresholds as the paper (≥ 7 = positive, 5–6 = neutral, ≤ 4 = negative), our within-2-star accuracy of ~80% suggests the model is achieving broadly comparable coarse-grained sentiment accuracy to their 75% baseline, while simultaneously solving a much harder fine-grained problem.

12. Generative-AI Task

The generative AI component extends the pipeline beyond classification: for each patient review, a large language model generates a concise summary (maximum 10 words) and assigns a binary sentiment label (positive or negative). Two distinct prompting strategies were designed and evaluated to demonstrate prompt engineering best practices.

12.1 Choice of generative model

I used Llama 3.2 (3B) served locally through Ollama, called via its OpenAI-compatible chat endpoint.

05_generative_ai.ipynb — LLM call

```
def call_local_llm(prompt, system_instruction=None):
    messages = []
    if system_instruction:
        messages.append({"role": "system", "content": system_instruction})
    messages.append({"role": "user", "content": prompt})
    response = requests.post(OLLAMA_URL, json={
        "model": "llama3.2:3b",
        "messages": messages,
        "temperature": 0.0,      # deterministic -> reproducible, reliable JSON
        "stream": False})
    return response.json()["choices"][0]["message"]["content"]
```

WHY A LOCAL MODEL, AND WHY TEMPERATURE 0

A local LLM was chosen for three reasons that matter in a medical context: privacy (patient reviews never leave the machine), zero per-call cost (so I can process the whole test set without rate limits), and reproducibility (the model is pinned, not a moving cloud endpoint). I set temperature = 0 deliberately: this is an extraction task, not a creative one. I want the same review to always yield the same summary and sentiment, and deterministic decoding also makes the model far more likely to emit clean, parseable JSON.

12.2 Prompt engineering

I designed two genuinely different prompting strategies, embodying different prompt-engineering best practices, and ran the full pipeline with each.

Alternative A: Zero-shot structured prompt

A single, explicit instruction that names the role, the task, the constraints, and the exact output schema with no examples.

```
You are analyzing a patient medical review. Your task is to extract an
ultra-short summary and a binary sentiment.

Review to analyze: "{review}"

Return your response exactly as a valid JSON object with these two keys:
- "summary": A summary that is strictly a MAXIMUM of 10 words.
- "sentiment": Output exactly either "positive" or "negative".
```

Alternative B: Few-shot, system-instructed prompt

This strategy splits the instruction into a persistent system role (defining the assistant as a clinical extraction specialist with hard constraints) and a user prompt that supplies two worked examples before the target review, using XML-style tags to delimit sections.

```
SYSTEM: You are a specialized clinical data extraction assistant...
1. The summary field must be 10 words or fewer.
2. The sentiment field must be exactly 'positive' or 'negative'.
3. Respond ONLY with a valid JSON block... no markdown, no backticks.

USER:
<examples>
Review: "This medication completely changed my life..."
Output: {"summary": "Life-changing drug eliminated all chronic symptoms quickly...",
        "sentiment": "positive"}
Review: "Absolutely horrible experience. I suffered terrible nausea..."
Output: {"summary": "Severe side effects including intense nausea and headaches.",
        "sentiment": "negative"}
</examples>
<target_review>{review}</target_review>
Output JSON:
```

Best practice	How Alternative A uses it	How Alternative B uses it
Role assignment	Inline ("You are analyzing...")	Dedicated persistent system role
Explicit constraints	✓ (max 10 words, exact labels)	✓ enumerated as numbered rules
Output schema	✓ JSON keys specified	✓ JSON, reinforced by examples
Few-shot examples	✗ None (zero-shot)	✓ Two contrasting worked examples
Structural delimiters	Quotation marks	✓ XML-style <code><tags></code>

WHY THESE TWO, AND WHAT EACH TESTS

Alternative A tests how well a small model follows a clean, well-specified

instruction on its own. Alternative B applies the heavier machinery that the prompt-engineering literature recommends for small models: few-shot examples anchor the output format and the desired summary style, while the system role and XML delimiters reduce the chance of the model drifting off-format. For a 3B-parameter model – which is more prone to format errors than a frontier model, I expected B to produce more consistent, constraint respecting output, which is why both are delivered: A as the clean baseline, B as the engineered improvement.

12.3 Robust parsing & output files

Small local models occasionally wrap JSON in markdown fences or emit a malformed object. The pipeline strips code fences, parses the JSON, and retries up to three times before recording a graceful error, so a single bad generation never crashes a run over the whole test set.

05_generative_ai.ipynb — defensive parsing

```
raw_text = re.sub(r'```json|```', '', raw_text).strip() # remove stray fences
data = json.loads(raw_text) # parse
summaries.append(data.get("summary", "Error parsing summary"))
sentiments.append(data.get("sentiment", "Error parsing sentiment"))
# wrapped in a 3-attempt retry loop with explicit error logging
```

Each run produces one spreadsheet with exactly the requested columns. The two deliverables are:

File	Prompt strategy
<code>alternative_a_zero_shot.xlsx</code>	Alternative A (zero-shot structured)
<code>alternative_b_few_shot.xlsx</code>	Alternative B (few-shot, system-instructed)

14. Conclusion

This project delivers a complete, honestly-evaluated solution to all three parts of the assignment. I built a Bidirectional-LSTM rating predictor, confronted the dataset's severe class imbalance head-on with class weighting, and made the deliberate, well-reasoned choice to favour a model with stronger macro-F1 over one with a flattering but hollow accuracy. I evaluated it with the metrics appropriate to an imbalanced, ordinal target and explained why each was chosen.

The interpretability study, using LIME and a confidence analysis, demonstrates that the model learns authentic clinical sentiment vocabulary and that its errors

are mostly local and often low-confidence, the behaviour we want from a trustworthy medical model. Placed on an equal footing with the reference paper's binary sentiment task, my model is competitive while additionally solving the harder fine-grained problem and adding explanations the original work lacked.

Finally, the generative-AI track shows command of modern LLM tooling and prompt engineering: a privacy-preserving local model, deterministic decoding, two principled prompt designs (zero-shot vs few-shot), robust parsing, and two clean Excel deliverables. Taken together, the work demonstrates not just the ability to run models, but the judgement to choose, evaluate, explain and critique them.

Reference

Gräßer, F., Kallumadi, S., Malberg, H., & Zaunseder, S. (2018). Aspect-Based Sentiment Analysis of Drug Reviews Applying Cross-Domain and Cross-Data Learning. Proceedings of the 2018 International Conference on Digital Health (DH '18), 121–125. ACM. Dataset mirrored on the UCI Machine Learning Repository and the Kaggle "KUC Hackathon Winter 2018".